

NYCU DL Lab6 – Generative Models

Student ID: 313553014 Name: 廖怡誠

1. Introduction

在 Lab6 中，需要實作 conditional denoising diffusion probabilistic model (DDPM)，並應用於通過 multi-label conditions 產生合成圖片的任務。在這個任務中，我們會有文字的 label 與對應的圖片，例如，label 為”red sphere”、”yellow cube”則圖片上也會相對應的物件，使用這些資料訓練 DDPM，使其在 inference 的階段能夠只給定文字就能夠生成 normal gaussian distribution 中生成相對應的圖片，最後，則會再使用 pre-trained 的分類器評估模型的效能。

2. Implementation Details

A. Describe how you implement your model, including your choice of DDPM, noise schedule.

我將分成 Data loader、Training code、Module、Testing code 三個部分進行說明。

● Data loader

此部分(圖 1.)說明我如何處理 dataset，本次任務的 condition label 是文字，共有 24 種類別，為了能夠更好的傳入到 model 中訓練，我將文字轉換成 one-hot code，舉例來說，有”red sphere”與”yellow cube”的 label，那除了這兩個 label 對應到的位置為 1 之外皆為 0，維度為(batch size, 24)，而圖片的部分則會將其 resize 到規定的 64 * 64 與 normalization 的 data augmentation。

● Training code.

此部分(圖 2、圖 3、圖 4、圖 5)說明我如何進行訓練以及使用的超參數，首先，建立 Class Trainer，其中包含 training 與 validation (此部分會在 testing code 說明)的程式碼，第一步是獲取 training 與 validation(new_test.json)的 data loader(圖 2)，此處的 validation 主要功能是通過生成的圖片來觀察模型的訓練效果，圖 3 中有此訓練中所使用的超參數，其中的 noise scheduler 使用 DDPM Scheduler，會以線性的方式在每個 time step 中加入 noise。主要的訓練部分(圖 4)，從 Dataloader 中取出 image 與 label，並產生與 image 大小相同的 noise，其會從 normal distribution 隨機取出，接著加入到隨機的

time step 中以增加模型泛化的能力，再傳入至模型就可以得到一個範圍在 normal distribution 中的 noise，我們期望此 noise 能與前面隨機產生的 noise 越接近越好，最後進行 backward 與儲存模型。

```
class LoadDataset(torchData):
    def __init__(self, data_path, json_path, objects_path, mode="train", partial=1.0):
        super().__init__()

        self.data_path = data_path
        self.mode = mode
        self.partial = partial

        self.transform = transforms.Compose([
            transforms.Resize((64, 64)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]),
        ])

        with open(json_path, 'r') as f:
            self.label_json_file = json.load(f)
            if self.mode == "train":
                self.imgs, self.labels = list(self.label_json_file.keys()), list(self.label_json_file.values())
            elif self.mode in ["test", "new_test"]:
                self.labels = self.label_json_file

        with open(objects_path, "r") as f:
            self.obj_json_file = json.load(f)

        self.label_one_hot = torch.zeros(len(self.labels), len(self.obj_json_file))

        for i, label in enumerate(self.labels):
            for j in label:
                self.label_one_hot[i][self.obj_json_file[j]] = 1

    def __len__(self):
        return int(len(self.labels) * self.partial)

    def __getitem__(self, idx):
        if self.mode == "train":
            img_path = os.path.join(self.data_path, self.imgs[idx])
            return self.transform(imgloader(img_path)), self.label_one_hot[idx]
        elif self.mode in ["test", "new_test"]:
            return self.label_one_hot[idx]
```

圖 1. Data loader 程式碼

```
class Trainer():
    def __init__(self, args):
        train_dataset = LoadDataset(data_path= args.data_path,
                                     json_path= args.train_json_path,
                                     objects_path= args.objects_file_path)
        self.train_loader = DataLoader(train_dataset,
                                       batch_size= args.batch_size,
                                       num_workers= args.num_workers,
                                       pin_memory= True,
                                       shuffle= True)

        val_dataset = LoadDataset(data_path= args.data_path,
                                    json_path= args.test_json_path,
                                    objects_path= args.objects_file_path,
                                    mode= "new_test")
        self.val_loader = DataLoader(val_dataset,
                                    batch_size= 32,
                                    num_workers= args.num_workers,
                                    pin_memory= True,
                                    shuffle= False)
```

圖 2. Trainer 的 Data loader 程式碼

```
# create a scheduler, adding small amount of noise for every time steps
self.noise_scheduler = DDPMSScheduler(num_train_timesteps= 1000)
self.model = CondDDPM().to(args.device)
self.loss_fn = nn.MSELoss()
self.optim = torch.optim.Adam(self.model.parameters(), lr= args.learning_rate)
self.epochs = args.epochs
self.batch_size = args.batch_size
self.device = args.device
self.save_img_per_epoch = args.save_img_per_epoch

self.prev_epochs = 0
self.history = {"train_loss": list()}
self.best_loss = np.inf
if args.pre_train is not None:
    checkpoint = torch.load(args.pre_train)
    self.model.load_state_dict(checkpoint['model_state_dict'])
    self.optim.load_state_dict(checkpoint['optimizer_state_dict'])
    self.prev_epochs = checkpoint['epoch']
    self.history = checkpoint['history']
    self.best_loss = checkpoint['best_loss']
```

圖 3. 超參數程式碼

```

def train(self):
    for epoch in range(self.prev_epochs, self.epochs):
        self.model.train()
        train_batch_loss = []
        pbar = tqdm(enumerate(self.train_loader))

        for i, (img, label) in pbar:
            img = img.to(self.device)
            label = label.to(self.device)
            noise = torch.randn_like(img)
            time_steps = torch.randint(0, 999, (img.shape[0],)).long().to(self.device)
            noisy_x = self.noise_scheduler.add_noise(img, noise, time_steps)
            pred = self.model(noisy_x, time_steps, label)
            # how close is the output to the noise
            loss = self.loss_fn(pred, noise)
            self.optim.zero_grad()
            loss.backward()
            self.optim.step()

        train_batch_loss.append(loss.item())
        pbar.set_description_str(f"[Train] epoch: {epoch + 1} / {self.epochs}, iter: {i + 1} / {len(self.train_loader)}, loss: {np.mean(train_batch_loss)}")

    # save model
    train_loss = np.mean(train_batch_loss)
    if self.best_loss > train_loss:
        self.best_loss = train_loss
        torch.save({
            'model_state_dict': self.model.state_dict(),
            'epoch': epoch,
            'history': self.history_loss,
            'best_loss': self.best_loss,
            # 'scheduler_state_dict': self.noise_scheduler.state_dict(),
            'optimizer_state_dict': self.optim.state_dict(),
        }, f"result/best.ckpt")

    self.history["train_loss"].append(train_loss)

```

圖 4. 主要 training 程式碼

```

if epoch % self.save_img_per_epoch == 0:
    self.model.eval()

    with torch.no_grad():
        pbar = tqdm(enumerate(self.val_loader))
        for i, label in pbar:
            label = label.to(self.device)
            # sample from normal distribution
            img = torch.randn(label.shape[0], 3, 64, 64).to(self.device)
            for t in self.noise_scheduler.timesteps:
                residual = self.model(img, t, label)
                img = self.noise_scheduler.step(residual, t, img).prev_sample
            pbar.set_description_str(f"[Val] epoch: {epoch + 1} / {self.epochs}, T: {t}")

    self.save_img(img, epoch, i)

```

圖 5. validation 程式碼

● Module

模型架構使用一層的 linear layer 對 conditional label 做類似 embedding 的操作，接著使用 Diffuser API 中的 UNet2DModel 作為 DDPM 中關鍵的 UNet，其使用 self-attention 作為 down/up sampling 的主要 Block，與使用 cross-attention 的 UNet2DConditionModel 相比，運算量較小，且在此 case 中有能夠有不錯的效果，將於後續的實驗說明。在 UNet2DModel 傳入的參數中，我將 down/up sampling 的層數調整至六層，Channel size 依序為[128, 128, 256, 256, 512, 512]，且為了降低運算量只在其中兩層使用 attention 機制，除此之外，依照預設傳入 conditional label，其會自動幫我做 embedding，但因為我已經使用 one hot code 且有使用額外的 linear layer 對其運算，因此我將 class_embed_type 設定成 identity，此 type 不會對 conditional label 操作。

```

class CondDDPM(nn.Module):
    def __init__(self, num_classes= 24, dim= 512):
        super().__init__()
        self.UNet = UNet2DModel(
            sample_size= 64,
            in_channels= 3,
            out_channels= 3,
            layers_per_block= 2,
            block_out_channels= (dim // 4, dim // 4, dim // 2, dim // 2, dim, dim),
            down_block_types= (
                "DownBlock2D",
                "DownBlock2D",
                "AttnDownBlock2D",
                "DownBlock2D",
                "AttnDownBlock2D",
                "DownBlock2D",
            ),
            up_block_types= (
                "UpBlock2D",
                "AttnUpBlock2D",
                "UpBlock2D",
                "AttnUpBlock2D",
                "UpBlock2D",
                "UpBlock2D",
            ),
            class_embed_type = "identity"
        )

        self.label_embedding = nn.Linear(num_classes, dim)

    def forward(self, x, t, labels):
        """
        x: noisy x,
        t: time steps,
        class_labels: y
        """
        embedding_label = self.label_embedding(labels)
        return self.UNet(x, t, embedding_label).sample

```

圖 6. Conditional DDPM 程式碼

- Testing code

此部分(圖 7)說明如何生成出指定的圖片，首先會建立 Class Tester，並獲取 test.json 與 new_test.json 的 data loader，並設置與 training 相同的 DDPM Scheduler，testing 主要的任務是，從 normal distribution 中得到一個 noise 並給定任意的文字 label(例如: "red sphere"、"yellow cube")，則模型需要通過 denoise 的方式生成出一張有紅色圓球與黃色方塊的圖片，最後會使用助教指定的 make_grid 函式儲存圖片，而在我的程式碼中，我會連續測試 test 與 new_test 的 data。

```

class Tester():
    def __init__(self, args):
        test_dataset = LoadDataset(data_path= "",
                                   json_path= args.test_json_path,
                                   objects_path= args.objects_file_path,
                                   mode= "test")

        self.test_loader = DataLoader(test_dataset,
                                      batch_size= args.batch_size,
                                      num_workers= args.num_workers,
                                      pin_memory= True,
                                      shuffle= False)

        new_test_dataset = LoadDataset(data_path= "",
                                        json_path= args.test_json_path,
                                        objects_path= args.objects_file_path,
                                        mode= "new_test")

        self.new_test_loader = DataLoader(new_test_dataset,
                                          batch_size= args.batch_size,
                                          num_workers= args.num_workers,
                                          pin_memory= True,
                                          shuffle= False)

        self.device = args.device
        # model for training
        self.train_model = DDPM().to(self.device)
        checkpoint = torch.load(args.pre_train, map_location=self.device)
        self.train_model.load_state_dict(checkpoint["model_state_dict"])
        # model for evaluation
        self.eval_model = evaluation_model(self.device)
        self.noise_scheduler = DDPMScheduler(num_train_timesteps= 1000)

        self.de_normalize = transforms.Normalize(mean= [-1.0, -1.0, -1.0], std= [2.0, 2.0, 2.0])

```

圖 7. Tester 初始化程式碼

```

def inference(self):
    self.train_model.eval()
    torch.manual_seed(3014)
    with torch.no_grad():
        pbar = tqdm(enumerate(self.test_loader))
        for i, label in pbar:
            label = label.to(self.device)
            img = torch.randn(label.shape[0], 3, 64, 64).to(self.device)
            for t in self.noise_scheduler.timesteps:
                residual = self.train_model(img, t, label)
                img = self.noise_scheduler.step(residual, t, img).prev_sample
                pbar.set_description_str(f"[Test] T: {t}")
            acc = self.eval_model.eval(img, label)
            de_img = self.de_normalize(img)
            self.pltImageGrid(de_img, "./result/test_grid.png")
            print("\ntest acc ", acc)

    with torch.no_grad():
        pbar = tqdm(enumerate(self.new_test_loader))
        for i, label in pbar:
            label = label.to(self.device)
            img = torch.randn(label.shape[0], 3, 64, 64).to(self.device)
            for t in self.noise_scheduler.timesteps:
                residual = self.train_model(img, t, label)
                img = self.noise_scheduler.step(residual, t, img).prev_sample
                pbar.set_description_str(f"[Test] T: {t}")
            acc = self.eval_model.eval(img, label)
            de_img = self.de_normalize(img)
            self.pltImageGrid(de_img, "./result/new_test_grid.png")
            print("\nnew test acc ", acc)

def pltDenoiseProcess(self, images, path):
    plt.imshow(torchvision.transforms.ToPILImage()(torchvision.utils.make_grid(images, nrow=10)))
    plt.savefig(path)

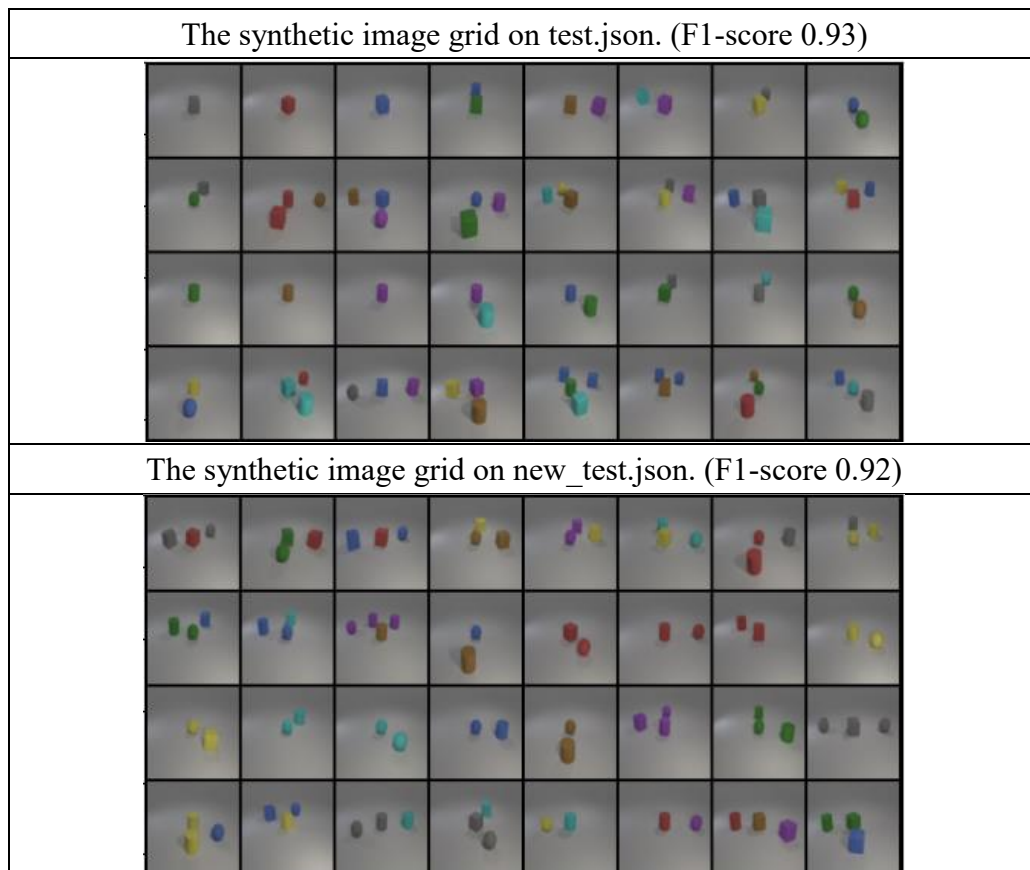
def pltImageGrid(self, images, path):
    plt.imshow(torchvision.transforms.ToPILImage()(torchvision.utils.make_grid(images, nrow=8)))
    plt.savefig(path)

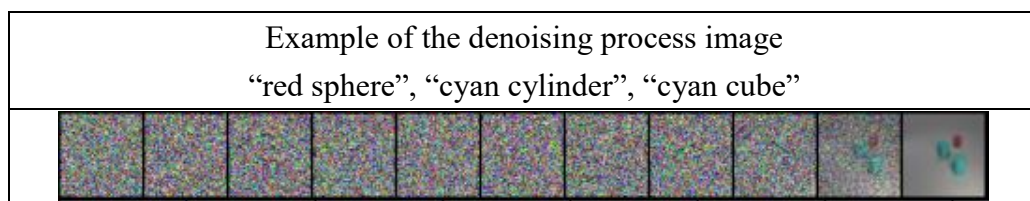
```

圖 8. 主要的 testing 程式碼

3. Results and discussion

A. Show your synthetic image grids and a denoising process image





B. Discussion of your extra implementations or experiments

在本次作業中，我嘗試不同的架構與方法來達到最佳化，輸出的圖片皆是使用 new_test.json 測試，各版本分別如下：

● Basic Version

參考 Diffuser tutorial 撰寫，使用 3 層的 up/down sampling block，其中兩層為 attention，channel size 為[32, 64, 64]，此版本的結果並不好，即使已經訓練 200 epochs，但 loss 仍然卡在 0.0020 左右，輸出的圖片也沒辦法是白色背景而是鮮豔的色彩，圖 9 所示。我認為是層數使用太少，使模型沒辦法有效學習色彩的資訊。

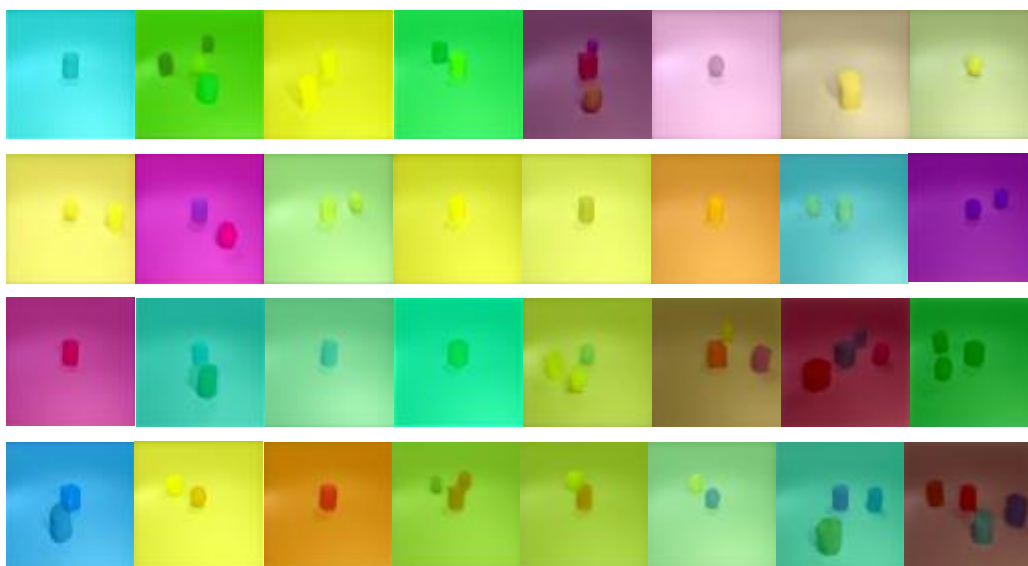


圖 9. Basic Version 結果圖

● Multi-Layer Version

修改上一個版本的缺點，我延續基礎架構，並改成 4 層的 up/down sampling block，其中三層為 attention，channel size 為[64, 128, 128, 256]，並訓練 300 epochs 左右，其結果的圖片背景已經不會出現鮮豔的色彩，但 ground truth 對比之後，會發現顏色、形狀和數量還是會有錯誤(圖 10)，像是右下角的”green cyclinder”的形狀還沒有很完整或是有四個物體在圖片中，

訓練 loss 大約卡在 0.10 左右，測試分數大約 70%。

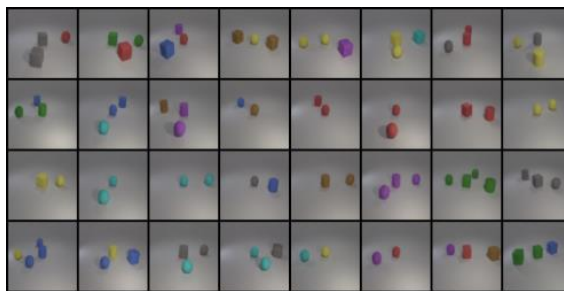


圖 10. Multi-Layer Version 結果圖

- Large-Layer Version

由於前一個版本的效果還可以再改善，因此我先嘗試將 UNet2DModel 改成 UNet2DConditionModel，但由於模型大小的限制，沒辦法如同前面的作法加深模型，只使用預設的 4 層的 up/down sampling block，其中三層為 attention，channel size 為[64, 128, 128, 256]，模型初始的 Loss 相較前兩個來的低，但由於訓練一個需要花費 30 分鐘左右，因此只簡單測試效果就停止訓練了，我認為若有足夠的時間與資源，cross-attention 的效果會比 self-attention 好。

- Deep-Layer Version (Best Version)

由於 Large-Layer Version 受限於資源，因此重新使用 UNet2DModel，並在加深 block 的層數，並加大 channel size，使用 6 層的 up/down sampling block，其中兩層為 attention，channel size 為[128, 128, 256, 256, 512, 512]，效果如預期的一樣，相比 Multi-Layer 的效果更好一點。

最後圖 11、圖 12 是上述方法的 Loss 曲線圖，由於 Large Version 訓練的 epoch 數不足，所以不放上來比較，只比較 Basic Version、Multi-Layer Version、Deep-Layer Version，可以觀察到加深模型的深度與 channel 數量對於提升效能是有不錯的成效。

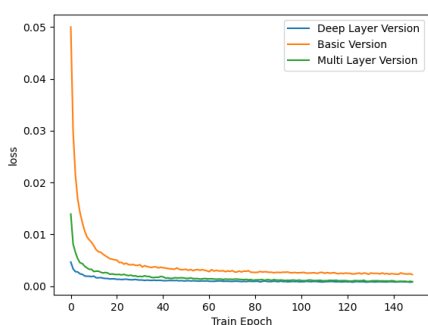


圖 11.完整 Loss

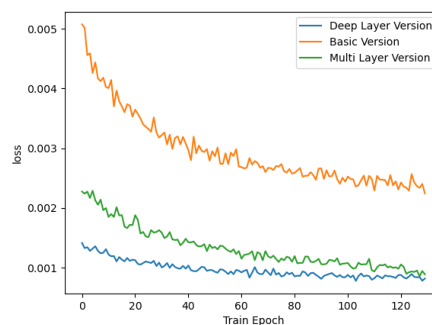


圖 12 尾端清楚 Loss

4. Classification accuracy on test.json and new test.json

A. Show your accuracy screenshots

```
[Test] T: 0: 0it [10:12, ?it/s]
test acc 0.9305555555555556
[Test] T: 0: 1it [10:17, 617.93s/it]
[Test] T: 0: 0it [11:25, ?it/s]
new test acc 0.9285714285714286
[Test] T: 0: 1it [11:34, 694.87s/it]
```

圖 13 accuracy screenshot

B. The command for inference process for both testing data

我將 new_test.json 與 test.json 寫在同一份 test.py 中，所以只須執行一次即可得到兩個測試的結果，而 test.py 需要輸入的參數如下表，助教填入 test、new_test、objects 三個檔案的路徑，以我的環境來說，我將 file 資料夾放在最外層，因此填入的路徑為 ./file/{file_name}，而權重(pre-train)的部分則是依據助教放置權重的位置填入，並且我有分別設置 random seed 來固定在(Ubuntu22.04)的輸出結果。

另外 evaluator.py 中的 checkpoint.pth，權重需要根據助教放的位置做設定。

inference	
device	cuda:0
test_json_path	./file/test.json
new_test_json_path	./file/test.json
objects_file_path	./file/objects.json
batch_size	32
num_workers	4
pre-train	依據助教放權重的位置
示意指令	
python test.py --test_json_path {} --new_test_json_path {} \ --objects_file_path {} --pre-train {}	

```
(GPU) C:\Users\yicheng\Desktop\DL2024\lab6>python test.py --pre-train best_deep.ckpt
```


5. 測試環境

我的實驗 cuda 版本如下圖，且在檔案中有附上環境中的 library，torch 版本為 2.4.0

```
(DLP) yicheng@user-WS-ESC700-G4-WS880T-E700-G4:~/NYCU_DeepLearning2024/lab6$ nvidia-smi
Fri Aug 23 23:23:22 2024

+-----+
| NVIDIA-SMI 525.147.05   Driver Version: 525.147.05   CUDA Version: 12.0   |
+-----+-----+
| GPU  Name      Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|                               |                    |                MIG M. |
+-----+-----+
|  0  NVIDIA GeForce ...  Off | 00000000:65:00:0 |      0%      N/A |
| 30%   34C   P8     4W / 250W | 18MiB / 11264MiB |             Default |
|                               |                    |                N/A |
+-----+-----+
|  1  NVIDIA GeForce ...  Off | 00000000:B3:00:0 |      0%      N/A |
| 30%   32C   P8    21W / 250W |  8MiB / 11264MiB |             Default |
|                               |                    |                N/A |
+-----+-----+

+-----+
| Processes: |
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |
|      ID    ID              |              |                      Usage    |
+-----+-----+
|  0     N/A   N/A       1810     G   /usr/lib/xorg/Xorg             9MiB |
|  0     N/A   N/A       1975     G   /usr/bin/gnome-shell           3MiB |
|  1     N/A   N/A       1810     G   /usr/lib/xorg/Xorg             4MiB |
+-----+-----+
```